

Gamblang

1. Architettura

Gamblang è un linguaggio imperativo con supporto nativo a costrutti probabilistici e stocastici.

Il compilatore traduce il codice sorgente in C e delega la compilazione finale a GCC.

Il processo è suddiviso nelle seguenti fasi:

1. Parsing del sorgente.
2. Costruzione dell'AST (Abstract Syntax Tree).
3. Analisi semantica (Type Checking e Symbol Table).
4. Generazione del codice C (Code Generation).
5. Compilazione del file C tramite GCC in modo seamless.

Il compilatore è implementato interamente in Python.

2. Parsing

Il parsing è affidato alla libreria **Lark** con grammatica **LALR(1)**, garantendo un'analisi sintattica deterministica e priva di ambiguità.

Whitespaces e Commenti

Spazi, tabulazioni e ritorni a capo non hanno significato sintattico e vengono ignorati dal parser. L'indentazione serve esclusivamente a migliorare la leggibilità del codice. I commenti sono supportati tramite il simbolo `#` (stile Python) e vengono scartati in fase di lexing.

Blocchi e Sintassi

Le strutture che introducono un blocco condizionale o iterativo (`if`, `loop`, `risky`) richiedono la parola chiave `do` per aprire il blocco e terminano con la parola chiave `end`. Le istruzioni singole (dichiarazioni, assegnamenti, stampe) terminano con il punto e virgola `;`.

Esempio:

```
if score > 10 do
  print score;
end
```

Espressioni

La precedenza degli operatori è definita gerarchicamente nella grammatica.

Moltiplicazione (`*`) e divisione (`/`) hanno precedenza maggiore rispetto ad addizione (`+`) e

sottrazione (-). Le parentesi tonde possono essere utilizzate per modificare l'ordine di valutazione.

3. Analisi Semantica

L'analisi semantica viene eseguita scorrendo l'AST (tramite il pattern *Interpreter / Visitor*) prima della generazione del codice.

Symbol Table

Il compilatore mantiene una tabella dei simboli ad unico scope globale contenente le variabili dichiarate e il relativo tipo. I tipi primitivi supportati sono tre:

- STEADY (Variabili deterministiche classiche)
- COIN (Variabili booleane basate su probabilità)
- GAMBLE (Variabili stocastiche con memoria storica)

Durante questa fase vengono rilevati e bloccati:

- Utilizzo di variabili non dichiarate.
- Dichiarazioni duplicate della stessa variabile.
- Assegnazioni incompatibili con il tipo della variabile (Type Checking).
- Uso di tipi COIN all'interno di espressioni aritmetiche.

Regole di assegnazione

Il linguaggio impone operatori visivamente distinti in base al tipo:

- Le variabili STEADY utilizzano l'operatore =.
- Le variabili GAMBLE utilizzano l'operatore <- (per indicare l'inserimento in una distribuzione).
- L'utilizzo dell'operatore errato genera un'eccezione a tempo di compilazione.

Funzione ask (Input)

La funzione ask consente di acquisire input da tastiera a runtime.

- Non può essere utilizzata su variabili di tipo COIN.
- Se applicata a una variabile GAMBLE, il compilatore verifica semanticamente che la variabile sia stata preventivamente dichiarata e inizializzata correttamente con l'operatore <-.

4. Generazione del Codice

Il backend converte l'AST in codice C. Ogni nodo dell'albero sintattico viene tradotto nel corrispondente costruito C.

Mappatura dei tipi deterministici

I valori numerici stabili (STEADY) vengono rappresentati tramite il tipo C:

double

Questa scelta uniforma la rappresentazione interna dei dati ed evita conversioni continue tra interi e virgola mobile.

Tipo GAMBLE (Motore Stocastico)

Le variabili GAMBLE vengono rappresentate nel C generato tramite una struttura dati dedicata:

```
typedef struct {  
    double history[100];  
    int count;  
} gamble_t;
```

- history contiene i valori assegnati alla variabile nel corso del tempo.
- count indica il numero di valori memorizzati.

Ogni assegnazione tramite `<-` aggiunge un nuovo elemento allo storico e incrementa il contatore.

Quando una variabile GAMBLE viene utilizzata in un'espressione, il valore restituito viene selezionato casualmente tra quelli presenti nello storico, garantendo una distribuzione discreta sui valori passati. La lettura viene tradotta nel seguente accesso C:

```
var.history[rand() % var.count]
```

5. Runtime Probabilistico

Le funzionalità probabilistiche del linguaggio sono implementate tramite il generatore pseudo-casuale della libreria standard C (`<stdlib.h>`).

All'inizio della funzione main, viene eseguita l'inizializzazione del seed:

```
srand(time(NULL));
```

I confronti probabilistici utilizzano valori generati tramite la formula:

```
(double)rand() / RAND_MAX
```

Il cui risultato appartiene all'intervallo $[0.0, 1.0]$.

Tipo COIN

Una variabile COIN rappresenta una probabilità (es. 0.75).

Durante la valutazione, viene confrontata con un valore casuale generato a runtime per risolversi in un intero 1 (Vero) o 0 (Falso).

Blocco risky

Il blocco risky esegue il proprio contenuto solo se la condizione probabilistica specificata ha successo. Permette anche un ramo alternativo or do.

Esempio:

```
risky (0.25) do
  print "successo al 25%";
or do
  print "fallimento al 75%";
end
```

Normalizzazione delle probabilità (Clamping)

Per evitare errori matematici a runtime, le espressioni probabilistiche (usate in coin e risky) vengono limitate in modo sicuro all'intervallo [0.0, 1.0].

Il codice generato include automaticamente la seguente macro:

```
#define CLAMP_PROB(p) ((p) > 1.0 ? 1.0 : ((p) < 0.0 ? 0.0 : (p)))
```

La macro viene applicata "silenziosamente" ai valori utilizzati dai costrutti probabilistici prima della loro valutazione.

6. Output Generato

Il compilatore produce un file C (output.c) completo e pronto per l'esecuzione.

Il file contiene:

1. Inclusioni delle librerie C necessarie (stdio.h, stdlib.h, time.h).
2. Dichiarazioni dei tipi runtime (gamble_t) e della macro CLAMP_PROB.
3. Inizializzazione stocastica (srand).
4. Traduzione delle dichiarazioni, delle variabili e delle strutture di controllo.
5. Formattazione pulita e indentata per facilitare il debug.